

Hyperparameter Tuning with Optuna: Bayesian Optimisation

Contents

1. Overview	2
2. Why grid search and random search are not enough	3
2.1 The problem: a random forest for placement	3
2.2 Grid search is expensive	3
2.3 Random search can miss the best combination	3
3. Bayesian search	4
3.1 Accuracy is a function of the hyperparameters	4
3.2 Learning the surface from the trials	4
3.3 Informed search versus blind search	5
3.4 The three searches on real data	6
4. Optuna's vocabulary	8
5. The Optuna workflow	8
5.1 The objective function	8
5.2 Creating the study and running the trials	9
5.3 The best trial and the final model	10
6. Samplers: Bayesian, random or grid	10
7. Visualising a study	11
8. Define-by-run: searching over algorithms	12
8.1 The problem: tuning several algorithms	12
8.2 A dynamic search space	13
8.3 Reading the trials as a table	14
9. Other features of Optuna	14
10. Summary	16
11. Key terms	16

1. Overview

Key point: Optuna is a hyperparameter tuning library; instead of trying combinations blindly, it uses the results of past trials to choose the next one.

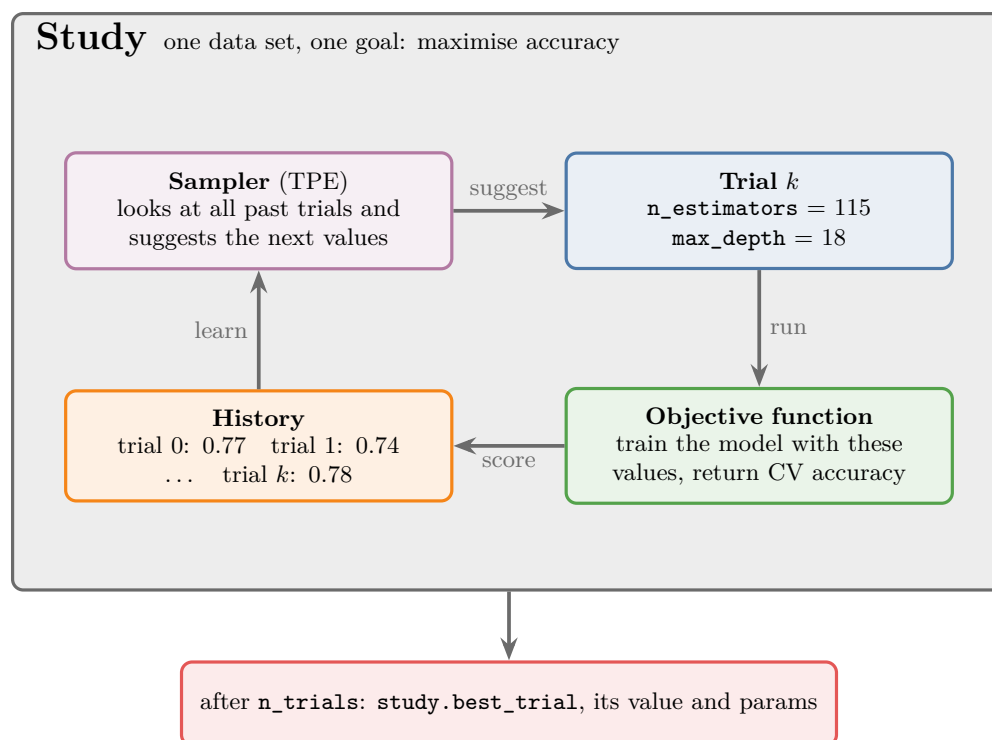


Figure 1: The Optuna loop: a sampler suggests values, a trial runs the objective function, the score joins the history, and the sampler learns from it

Optuna is an open-source Python framework for hyperparameter tuning, widely used in industry for both machine learning and deep learning. Its main strength is the kind of search it runs: **Bayesian optimisation**, which learns from every trial where good hyperparameters are likely to be (Figure 1).

This Note covers why grid and random search are not always enough, the idea of Bayesian search, Optuna’s vocabulary and workflow, its samplers and plots, and its “define-by-run” search spaces.

Extra: Optuna is not installed in this project’s environment, so the Optuna code in this Note is shown in Python boxes but not run. The Notebook (`notebook.ipynb`) shows the same ideas with scikit-learn: grid and random search, a Bayesian search written by hand, and a define-by-run objective run through a small stand-in for Optuna’s `trial`. To use Optuna itself: `pip install optuna`.

2. Why grid search and random search are not enough

Key point: Grid search trains every combination, which becomes too slow; random search trains a few, which can miss the best one. Neither learns from its earlier trials.

Hyperparameter tuning (the [pipelines Note](#)) tries several hyperparameter values and keeps the best. So far we have done it with `GridSearchCV` (the [KNN Note](#), section 4.2) and `RandomizedSearchCV` (the [regression trees Note](#), section 7.2); the [random forest tuning Note](#) compares them on a forest.

2.1 The problem: a random forest for placement

Key point: We want the `max_depth` and `n_estimators` that give the highest accuracy, and we cannot know them in advance.

Take the placement data: predict from a student's CGPA and IQ whether they will be placed, with a random forest. We tune two hyperparameters ([random forest hyperparameters Note](#)):

- `max_depth`: how deep each decision tree may grow;
- `n_estimators`: how many trees the forest has.

First we choose a **search space**, the values to try, from intuition and domain knowledge: `max_depth` from 1 to 5, and `n_estimators` 50, 100, 150, 200 and 250. That makes a grid of $5 \times 5 = 25$ combinations.

2.2 Grid search is expensive

Key point: The number of models to train is the product of the number of values, so it explodes as we add values or hyperparameters.

Grid search trains a model for every one of the 25 combinations, scores each, and keeps the best. Given enough time it finds the best combination in the grid, but the cost grows fast:

- 20 values of `n_estimators` instead of 5: $20 \times 5 = 100$ models.
- A third hyperparameter, such as `min_samples_split` with 5 values: 500 models.
- A large dataset or a neural network that takes an hour to train: 500 hours.

2.3 Random search can miss the best combination

Key point: Random search trains only a fixed number of random combinations: cheap, but the best one may never be drawn.

Random search trains, say, 5 random combinations out of the 25 and keeps the best of those. It saves most of the computation. The price is that the best combination may simply not be among the 5 drawn, so we settle for a worse one.

Both searches share a deeper weakness: they are “blind”. The score of one trial never influences which combination is tried next.

3. Bayesian search

Key point: Treat accuracy as an unknown function of the hyperparameters, learn its shape from the trials so far, and try next where it is most likely to be high.

3.1 Accuracy is a function of the hyperparameters

Key point: Change `max_depth` or `n_estimators` and the accuracy changes, so accuracy = $f(\text{max_depth}, \text{n_estimators})$ for some unknown f .

If we change `max_depth` from 5 to 25, the accuracy changes; if we change `n_estimators` from 10 to 100, it changes too. So there is a relationship between the hyperparameters and the accuracy:

$$\text{accuracy} = f(\text{max_depth}, \text{n_estimators})$$

We do not know f . With two hyperparameters it is a surface over a flat grid: `max_depth` on one axis, `n_estimators` on another, accuracy as the height. If we knew the surface, we would read off its highest point, and with it the best hyperparameters.

3.2 Learning the surface from the trials

Key point: Every trial gives one point on the unknown surface; from these points the search guesses the surface and picks the most promising place to try next.

Each trial reveals one point of f . For example, `max_depth` = 5 and `n_estimators` = 100 with accuracy 75% is one point on the surface; `max_depth` = 10 and `n_estimators` = 150 with 69% is another.

To see this, drop `n_estimators` and keep one hyperparameter, so the surface becomes a curve (Figure 2):

1. Try `max_depth` = 5, 10 and 15: accuracies 85%, 65% and 84%. These are three points on the hidden curve.
2. From the three points, guess the whole curve, together with how unsure the guess is in each place: little near the points, a lot far from them.
3. Try next where the guess is high, or where it is very unsure and could be high.
4. Add the new point, update the guess, and repeat.

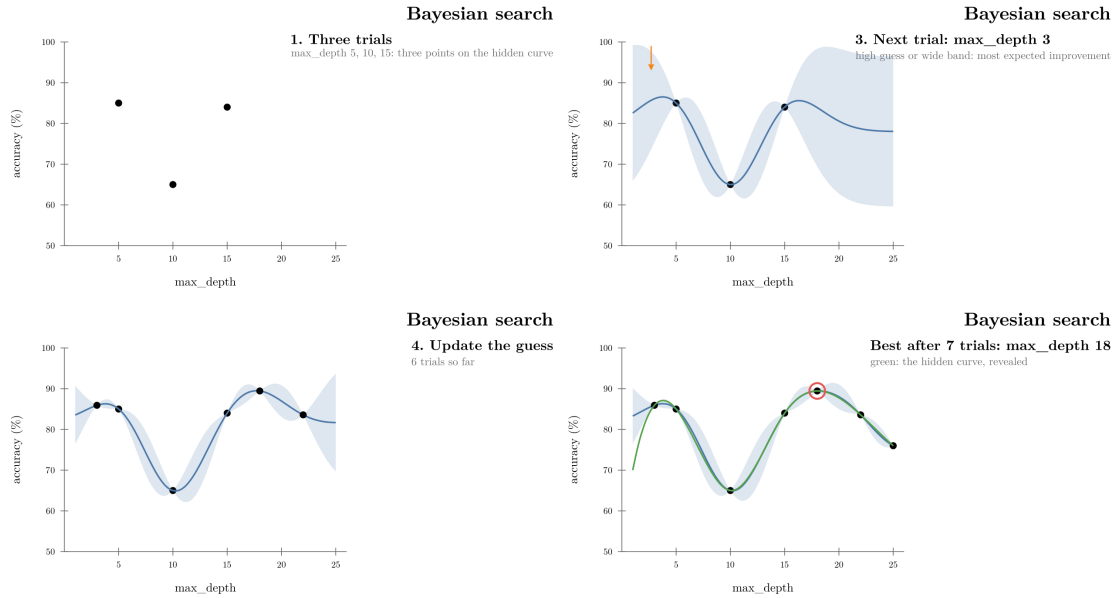


Figure 2: Bayesian search on one hyperparameter: three trials, a guess of the curve with its uncertainty (band), the next trial where the improvement could be largest, and the hidden curve revealed at the end

After a few rounds, the guess matches the hidden curve well near its top, and the best trial is close to the true maximum. Most trials end up in the promising region instead of being spread evenly.

3.3 Informed search versus blind search

Key point: Grid and random search ignore earlier results; Bayesian search uses every earlier result to choose the next trial.

	Grid search	Random search	Bayesian search
Next trial chosen	in a fixed order	at random	from the results so far
Uses earlier scores	no	no	yes
Trials needed	all combinations	as many as we allow	usually fewer for the same quality

Bayesian optimisation is the name of this informed approach. It is called Bayesian because it updates a belief about the unknown function with each new result, as Bayes' theorem updates a probability with new evidence (the [Bayes theorem Note](#)).

Extra: The two parts of Bayesian optimisation have names. The guess of the curve is the **surrogate model**: in Figure 2 a **Gaussian process**, a model that predicts a value and its uncertainty (a mean μ and a standard deviation σ) at every point. The rule

that picks the next trial is the **acquisition function**; a common one is the **expected improvement**.

Extra: Expected improvement, step by step.

1. **In words:** how much better than the best score so far we expect a point to be, averaging over the surrogate's uncertainty; a point counts as promising if its predicted mean is high, or if its uncertainty is wide enough that it might be high.
2. **Formula:** with best score so far f^* , predicted mean μ and standard deviation σ at a point, Φ the normal distribution's cumulative probability and ϕ its density:

$$z = \frac{\mu - f^*}{\sigma}, \quad \text{EI} = (\mu - f^*) \Phi(z) + \sigma \phi(z)$$

3. **Example:** $f^* = 0.785$, $\mu = 0.790$, $\sigma = 0.010$:

$$z = \frac{0.790 - 0.785}{0.010} = 0.5, \quad \Phi(0.5) = 0.6915, \quad \phi(0.5) = 0.3521$$

$$\text{EI} = 0.005 \times 0.6915 + 0.010 \times 0.3521 = 0.00346 + 0.00352 = 0.0070$$

The two halves are about equal: half of this point's appeal is its high mean, half its uncertainty.

3.4 The three searches on real data

Key point: On the diabetes data, our hand-written Bayesian search found a better cross-validated accuracy (0.797) than grid search (0.784) or random search (0.786), and spent 19 of its 50 trials in the best region.

The Notebook tunes a random forest on the Pima diabetes data (768 patients, 8 medical measurements, the [ROC Note](#)), split 70/30 into 537 training and 231 test rows. Every search maximises the same number, the mean 3-fold cross-validated accuracy ([pipelines Note](#), section 8):

- **Grid search:** `n_estimators` 50, 100, 150, 200 $\times$ `max_depth` 5, 10, 15, 20: 16 trials.
- **Random search:** 50 random combinations of `n_estimators` 50 to 200 and `max_depth` 3 to 20.
- **Bayesian search:** the same 50-trial budget and space; 10 random trials to start, then a Gaussian process with expected improvement chooses each trial.

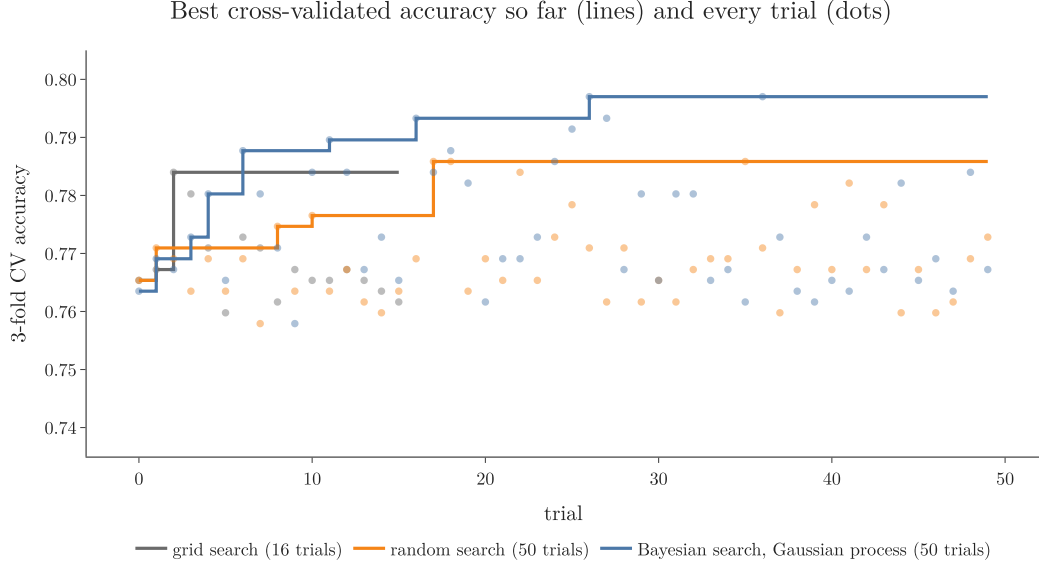


Figure 3: Each search’s best cross-validated accuracy so far (lines) and every trial (dots)

Search	Trials	Best <code>n_estimators</code>	Best <code>max_depth</code>	Best CV accuracy	Test accuracy
Grid	16	50	15	0.784	0.749
Random	50	53	12	0.786	0.740
Bayesian (ours)	50	53	8	0.797	0.740

Figure 3 shows the Bayesian search ahead of the other two from trial 6 onwards. Figure 4 shows why: after its random start, it kept returning to small forests of depth 8 to 9, where the scores were highest. 19 of its 50 trials lie in that region (`n_estimators` up to 80, `max_depth` 7 to 11), against 1 for the grid and 1 for random search.

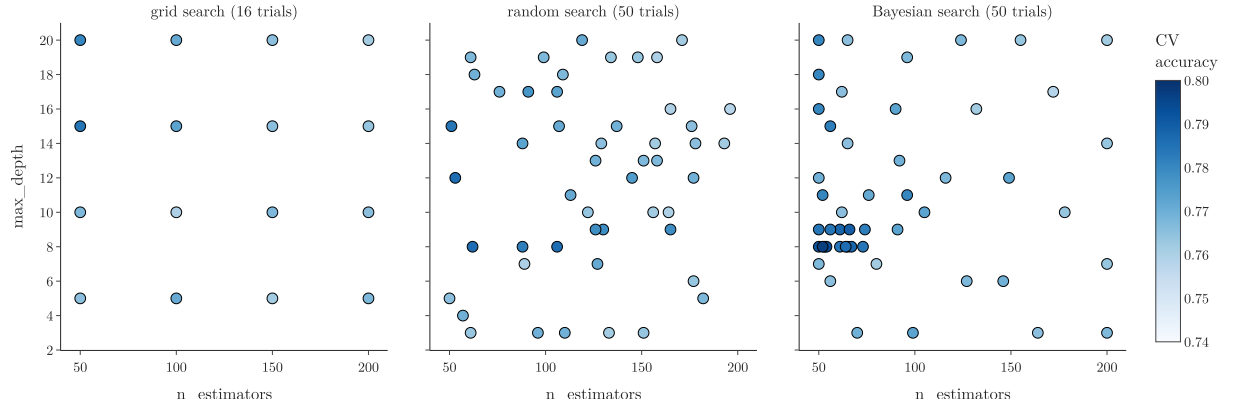


Figure 4: Where each search tried, coloured by cross-validated accuracy

Extra: The test accuracies are all about 0.74 to 0.75. On 537 rows, cross-validated accuracy is noisy, and a search that tries many nearby combinations partly finds the luckiest one. The gain from smarter search shows best on larger data and with more hyperparameters, where blind search wastes far more trials.

4. Optuna's vocabulary

Key point: A study is the whole tuning task; a trial is one run with one set of values; the objective function scores a trial; the sampler chooses the next values.

Five terms appear in every piece of Optuna code (Figure 1):

- **Study:** one optimisation session: one dataset, one algorithm (or several), the hyperparameters to tune and the goal. It is a collection of trials aimed at optimising the objective function.
- **Trial:** one run of the objective function with one set of hyperparameter values, for example `max_depth = 1` and `n_estimators = 50`.
- **Trial parameters:** the hyperparameter values used in a trial, for example `max_depth = 2` and `n_estimators = 100`.
- **Objective function:** the function we want to optimise. It takes a trial, builds and trains a model with that trial's values, and returns a score such as accuracy.
- **Sampler:** the algorithm that suggests which hyperparameter values to try in the next trial. Optuna's default is **TPE**, the Tree-structured Parzen Estimator, a form of Bayesian optimisation.

Extra: How TPE chooses. It sorts the past trials into a “good” group (by default about the best 10%) and a “bad” group. For each hyperparameter it estimates how its values are spread in each group, $\ell(x)$ for good and $g(x)$ for bad, and tries next the values where $\ell(x)/g(x)$ is highest: values common among good trials and rare among bad ones. Unlike a Gaussian process, it handles integer, categorical and conditional hyperparameters easily.

5. The Optuna workflow

Key point: Write an objective function, create a study, call `optimize`; the best trial holds the best score and values.

5.1 The objective function

Key point: Inside the objective, `trial.suggest_int` defines the search space and returns this trial's values; the function trains a model and returns its score.

The objective function does two things:

1. **Defines the search space** and takes this trial's values from it: `n_estimators` between 50 and 200, `max_depth` between 3 and 20.

2. **Runs the training:** builds a random forest with those values, scores it with 3-fold cross-validation, and returns the mean accuracy.

Python: The objective function (Optuna is not installed here, so not run).

```
import optuna
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score

def objective(trial):
    # 1. the search space: this trial's values
    n_estimators = trial.suggest_int("n_estimators", 50, 200)
    max_depth = trial.suggest_int("max_depth", 3, 20)
    # 2. the training
    model = RandomForestClassifier(
        n_estimators=n_estimators, max_depth=max_depth,
        random_state=42)
    scores = cross_val_score(model, X_train, y_train,
                             cv=3, scoring="accuracy")
    return scores.mean()

trial.suggest_int(name, low, high) returns a whole number between low and
high, both included, chosen by the sampler. Every value in the range is allowed,
so n_estimators can be 51 or 137, not only multiples of 50.
```

The trial object is where the intelligence is hidden: on each call, `suggest_int` asks the sampler, which looks at all earlier trials before answering.

Extra: The other `suggest_` methods: `trial.suggest_float("C", 1e-3, 1e3, log=True)` for decimal values (`log=True` spreads the tries evenly over orders of magnitude, as for regularisation strengths), and `trial.suggest_categorical("kernel", ["linear", "rbf"])` for a choice from a list.

5.2 Creating the study and running the trials

Key point: `create_study(direction="maximize")` for a score like accuracy, `"minimize"` for a loss; `optimize(objective, n_trials=50)` runs the trials.

Python: Two lines run the whole search.

```
study = optuna.create_study(
    direction="maximize",
    sampler=optuna.samplers.TPESampler())
study.optimize(objective, n_trials=50)

direction="maximize" because accuracy (or  $R^2$ ) should go up; for a loss such as log
loss or mean squared error we would write "minimize". TPE is the default sampler, so
sampler= can be left out. optimize calls objective 50 times, each time with a new
trial.
```

While it runs, Optuna prints one line per trial, for example `Trial 0 finished with value: 0.77` and `parameters: {'n_estimators': 69, 'max_depth': 8}`. Early trials are spread out; later ones gather in the regions that scored well.

5.3 The best trial and the final model

Key point: `study.best_trial.value` is the best score and `study.best_trial.params` its values; we retrain a model with them and test it once.

Python: Reading the result and testing the best model.

```
best = study.best_trial
best.value      # best mean CV accuracy
best.params     # e.g. {'n_estimators': 115, 'max_depth': 18}

model = RandomForestClassifier(**best.params,
                              random_state=42)

model.fit(X_train, y_train)
model.score(X_test, y_test)  # accuracy on the test set
**best.params unpacks the dictionary into keyword arguments: n_estimators=115,
max_depth=18.
```

The best score comes from cross-validation on the training data, so it is not the final word. The test accuracy of the retrained model is. In our own Bayesian search (section 3.4), the best combination scored 0.797 in cross-validation and 0.740 on the test set.

Extra: Better preprocessing would likely help more than more tuning on this data. Several columns use 0 for “not measured” (blood pressure, BMI, insulin), and imputing them ([imputation Note](#)) is a natural next step.

6. Samplers: Bayesian, random or grid

Key point: Swapping the sampler turns the same study into a TPE search, a random search or a grid search.

The objective function stays exactly the same; only the `sampler=` argument changes.

Python: Random search and grid search inside Optuna (not run here).

```
# random search: 50 random combinations
study = optuna.create_study(
    direction="maximize",
    sampler=optuna.samplers.RandomSampler(seed=42))
study.optimize(objective, n_trials=50)

# grid search: every combination of an explicit grid
space = {"n_estimators": [50, 100, 150, 200],
        "max_depth": [5, 10, 15, 20]}
```

```
study = optuna.create_study(
    direction="maximize",
    sampler=optuna.samplers.GridSampler(space))
study.optimize(objective, n_trials=16)
```

For `GridSampler`, the grid is written outside the objective, and the trial values come from it: $4 \times 4 = 16$ trials. With grid or random search inside Optuna, we no longer need scikit-learn's `GridSearchCV` or `RandomizedSearchCV`.

Our own runs of these two searches are in section 3.4: on this small dataset all three searches end up close, as is common.

Extra: Other samplers in `optuna.samplers` include `CmaEsSampler` (an evolution strategy for many numeric hyperparameters), `GPSampler` (Gaussian-process Bayesian optimisation, like ours) and `NSGAIISampler` (for studies with several objectives at once).

7. Visualising a study

Key point: `optuna.visualization` draws interactive Plotly charts of a finished study: the history, where the trials went, and which hyperparameters mattered.

Python: The five main plots (not run here; each returns a Plotly figure).

```
from optuna.visualization import (
    plot_optimization_history, plot_parallel_coordinate,
    plot_slice, plot_contour, plot_param_importances)

plot_optimization_history(study).show()
plot_parallel_coordinate(study).show()
plot_slice(study).show()
plot_contour(study).show()
plot_param_importances(study).show()
```

What each shows, with the figure of this Note that draws the same idea from our own search:

- **Optimisation history:** trial number against the objective value, with the best so far as a line (Figure 3). When the line stops rising, more trials are not paying off; this helps choose `n_trials`.
- **Parallel coordinates:** one vertical axis per quantity (the score and each hyperparameter) and one line per trial, coloured by score (Figure 5). Where many lines bunch on an axis, the sampler found a promising range.
- **Slice plot:** each hyperparameter separately against the score, one dot per trial. Dense regions show where the sampler spent its trials.
- **Contour plot:** two hyperparameters on the axes and the score as coloured contours, which shows at a glance where the best region of the search space is (Figure 4 shows the trial points this plot is built from).
- **Hyperparameter importances:** how much each hyperparameter affected the score in this study, as percentages that add up to 100%. For a random forest, `max_depth`

often matters much more than `n_estimators`.

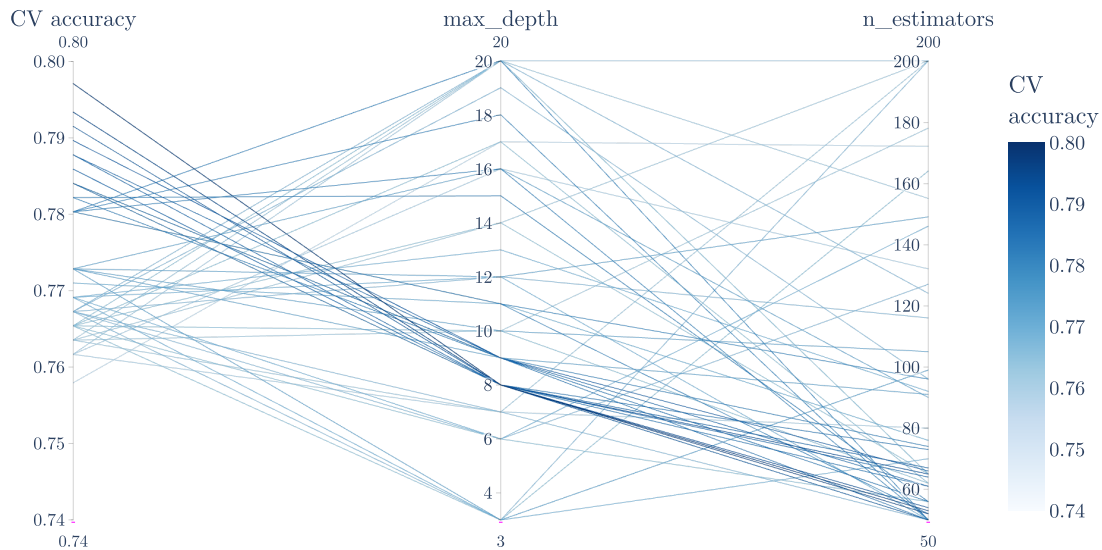


Figure 5: Parallel coordinates of our Bayesian search: each line is a trial; the darkest lines run through `max_depth` 8 and small `n_estimators`

In Figure 5, the darkest lines (the best trials) run from high accuracy through `max_depth` 8 to 9 and on to the bottom of the `n_estimators` axis: the same promising region as in Figure 4.

Extra: `optuna.visualization.matplotlib` has the same plots drawn with `matplotlib`; the default `optuna.visualization` versions use `Plotly` and are interactive.

8. Define-by-run: searching over algorithms

Key point: Because the search space is built while the objective runs, the algorithm itself can be a hyperparameter, and each algorithm can have its own hyperparameters.

8.1 The problem: tuning several algorithms

Key point: Tuning four algorithms one after another means four separate searches.

On a new dataset we rarely know which algorithm will do best: logistic regression, an SVM, a random forest, gradient boosting? Normally we tune each one separately and compare their best scores, which means four full searches.

8.2 A dynamic search space

Key point: Make `classifier` a categorical hyperparameter, then use `if/elif` to suggest only that classifier's hyperparameters.

In Optuna, the search space is not fixed in advance: it is created by the `suggest_` calls as the objective function runs. This is called **define-by-run**. So the first suggestion can pick the algorithm, and the later suggestions can depend on that choice. A search space whose hyperparameters depend on other hyperparameters is a **dynamic search space** (also called a conditional search space).

Python: One study over three algorithms (Optuna not run here; the Notebook runs this same function with a random stand-in for `trial`).

```
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

def objective(trial):
    name = trial.suggest_categorical(
        "classifier",
        ["SVC", "RandomForest", "GradientBoosting"])
    if name == "SVC":
        c = trial.suggest_float("C", 1e-3, 1e3, log=True)
        kernel = trial.suggest_categorical(
            "kernel", ["linear", "rbf"])
        model = make_pipeline(StandardScaler(),
                               SVC(C=c, kernel=kernel))
    elif name == "RandomForest":
        model = RandomForestClassifier(
            n_estimators=trial.suggest_int(
                "n_estimators", 50, 200),
            max_depth=trial.suggest_int("max_depth", 3, 20))
    else:
        model = GradientBoostingClassifier(
            n_estimators=trial.suggest_int(
                "gb_n_estimators", 50, 200),
            learning_rate=trial.suggest_float(
                "learning_rate", 0.01, 0.3, log=True),
            max_depth=trial.suggest_int("gb_max_depth", 2, 6))
    return cross_val_score(model, X_train, y_train,
                            cv=3).mean()

study = optuna.create_study(direction="maximize")
study.optimize(objective, n_trials=100)
```

```
study.best_trial.params
```

Each `if` branch suggests only its own algorithm's hyperparameters, so an SVC trial never draws a `max_depth`. The SVM gets a `StandardScaler` in a pipeline because it measures distances ([KNN Note](#), section 7.4).

The TPE sampler chooses the classifier like any other hyperparameter. After a few trials it notices which algorithm scores best and gives it most of the remaining trials, while also tuning that algorithm's own hyperparameters. One study replaces four.

8.3 Reading the trials as a table

Key point: `study.trials_dataframe()` gives one row per trial; grouping by the classifier column shows how often each algorithm was tried and how well it did.

Python: The trials as a pandas DataFrame.

```
df = study.trials_dataframe()
# columns: number, value, datetime_start, datetime_complete,
# duration, params_classifier, params_C, ..., state
df.groupby("params_classifier")["value"].agg(
    ["count", "mean", "max"])
```

A hyperparameter that a trial never suggested (such as `params_C` in a random forest trial) is NaN in that row.

The Notebook runs the objective of section 8.2 for 30 trials, with a stand-in `trial` that answers every `suggest_` call at random (like `RandomSampler`):

Classifier	Trials	Mean CV accuracy	Best CV accuracy
SVC	10	0.738	0.786 (linear kernel, C = 0.037)
Random forest	11	0.769	0.780
Gradient boosting	9	0.764	0.773

A random sampler splits the trials roughly evenly between the three algorithms. TPE instead shifts most of them to the algorithm that is scoring best, so the count column of its table is very uneven, and that algorithm gets tuned much more finely.

Extra: The SVC's low mean and high best go together: its score depends strongly on C and the kernel, so it has both the worst and the best trials. This is why comparing algorithms by their default settings can mislead.

9. Other features of Optuna

Key point: Optuna can spread trials over many machines, works with the main ML and DL libraries, and can stop bad trials early.

- **Distributed optimisation:** several processes or machines can run trials of the same study at once, sharing the results through a database, so the search finishes sooner.
- **Integrations:** Optuna works with scikit-learn, Keras, PyTorch, XGBoost, LightGBM and with MLflow (which records experiments).
- **Machine learning and deep learning alike:** the objective function can train anything that returns a score.

Python: A study stored in a database, so several processes can work on it (not run here).

```
study = optuna.create_study(
    study_name="rf-diabetes", direction="maximize",
    storage="sqlite:///study.db", load_if_exists=True)
study.optimize(objective, n_trials=50)
```

Running the same lines in several terminals at once makes each one add trials to the same study.

Extra: Pruning stops unpromising trials early. In a neural network trained for many epochs, the objective reports the score after each epoch with `trial.report(score, epoch)`; if `trial.should_prune()` returns `True` because the trial is doing worse than earlier ones at the same point, the objective raises `optuna.TrialPruned()` and the study moves on.

10. Summary

	Grid search	Random search	Bayesian search (Optuna TPE)
Chooses the next trial	in order	at random	from all earlier results
Cost	all combinations	<code>n_trials</code>	<code>n_trials</code>
Risk	too slow	misses the best	can follow noise on small data
In Optuna	<code>GridSampler</code>	<code>RandomSampler</code>	<code>TPESampler</code> (default)
Our best CV accuracy	0.784 (16 trials)	0.786 (50 trials)	0.797 (50 trials, Gaussian process)

- Bayesian optimisation treats the score as an unknown function of the hyperparameters, guesses it from the trials so far, and tries next where the improvement could be largest.
- Optuna's vocabulary: study, trial, trial parameters, objective function, sampler.
- The workflow: an objective with `trial.suggest_*`, then `create_study(direction=...)`, `optimize(objective, n_trials=...)`, `best_trial.params`, retrain and test.
- `optuna.visualization` gives Plotly charts: optimisation history, parallel coordinates, slice, contour, importances.
- Define-by-run lets one study choose the algorithm and its hyperparameters together.

11. Key terms

Term	Meaning
Optuna	A Python framework for hyperparameter tuning built around Bayesian optimisation
Search space	The ranges or lists of values each hyperparameter may take during tuning
Bayesian optimisation	Tuning that models the score as a function of the hyperparameters and uses all earlier trials to choose the next one
Surrogate model	The model of the unknown score function that Bayesian optimisation builds from the trials
Acquisition function	The rule that picks the next trial from the surrogate, such as expected improvement
Expected improvement	How much better than the best score so far a point is expected to be, given the surrogate's mean and uncertainty

Term	Meaning
Gaussian process	A model that predicts a value and its uncertainty at every point; a common surrogate
Study	In Optuna, one optimisation session: a collection of trials aimed at optimising the objective function
Trial	In Optuna, one run of the objective function with one set of hyperparameter values
Objective function	The function a search optimises: it takes a trial's values and returns a score
Sampler	In Optuna, the algorithm that suggests the next trial's hyperparameter values
TPE	Tree-structured Parzen Estimator, Optuna's default Bayesian sampler
Define-by-run	Building the search space while the objective function runs, so it can depend on earlier choices
Dynamic search space	A search space in which some hyperparameters exist only for some values of another, such as the algorithm
Pruning	Stopping an unpromising trial early, before its training finishes